

Wydział Informatyki Politechniki Białostockiej Przedmiot: Inżynieria Oprogramowania Kierunek: Informatyka Semestr: 4	Data: 10.06.2026
Sprawozdanie projektowe Temat projektu: Serwisi streamingowy Slop Stream Grupa: PS 10 Zespół: - Oliwier Zyskowski - Szymon Kalicki - Hubert Gudalewski	Prowadzący: mgr. Inż. Tomasz Hordjewicz

Skład zespołu oraz podział pracy:

Oliwier Zyskowski 117212 – Formatowanie sprawozdania, p.p. 2, 3, 4, 6, 9

Szymon Kalicki 117285 - Formatowanie sprawozdania, p.p. 2, 3, 4, 7, 10

Hubert Gudalewski 117340 - Formatowanie sprawozdania, p.p. 2, 3, 5, 8

Link do repozytorium projektu:

<https://github.com/hubertus1512/Projekt-I0-26>

1.Treść zadania projektowego:

Serwis streamingowy SlopStream:

Serwis streamingowy, gdzie twórcy treści mogą transmitować na żywo swoje treści dla widzów. Użytkownicy też mogą oglądać zapisane materiały albo prowadzić własne transmisje, sami zostając twórcami treści.

Użytkownicy mogą być gośćmi lub mieć konto. Konta się dzielą na użytkowników, streamerów i administratorów. Streamer i administrator dziedziczą klasę użytkownika. Użytkownik zawiera nazwę, uuid, e-mail i hasło. Streamer zawiera listę UUID (użytkowników), którzy go obserwują, i listę subskrybentów (też UUID).

Niezałogowany widz może przeglądać transmisje na żywo i zapisane filmy. W momencie wejścia użytkownika pobierana jest lista aktywnych streamów z bazy danych, i można je filtrować lub sortować przez statystyki, które też pobrano z powyższej bazy danych.

Niezałogowany użytkownik może się zarejestrować lub zalogować. Po wypełnieniu formularza rejestracyjnego serwer obliczeniowy z komponentem Authorization sprawdza, czy wszystko jest poprawne (czy taki użytkownik istnieje w bazie danych, czy hasło spełnia podstawowe wymogi, czy w bazie nie istnieje blokada adresu IP rejestrującego) po czym zapisuje do bazy danych informacje o użytkowniku i zwraca wiadomość końcową użytkownikowi.

W przypadku wypełnienia formularza logowania powyższy komponent sprawdza natomiast, czy użytkownik istnieje. W przypadku rejestracji po sprawdzeniu warunków wstępnych wysyłany też jest e-mail przez jeden z serwerów głównych.

Po rejestracji konto jest domyślnie w stanie nieaktywowanym. Podczas tego stanu nie może czatować z innymi streamerami, nie może pisać prywatnych wiadomości oraz nie może subskrybować twórców treści, ale może ich obserwować.

Po aktywacji maila, konto przechodzi w stan aktywny. Użytkownik może również zarządzać stanem swojego konta - w opcjach konta może usunąć swoje konto. Po zatwierdzeniu operacji konto staje się zdezaktywowane - ograniczenia te same, co konto w stanie nieaktywowanym. Użytkownik ma 30 dni na anulowanie usunięcia konta w opcjach konta. Po 30 dniach konto przechodzi w stan usunięty, ale nie jest fizycznie usunięte - jest dostępne przez rok w bazie danych w ramach bezpieczeństwa i zasad platformy. Po roku konto faktycznie jest permanentnie usuwane ze wszystkich baz.

Kanał może też być zweryfikowany po specjalnej weryfikacji przez obsługę. Takie konto ma wiele korzyści, np. możliwość bezpośredniego czatowania z administracją.

Po rejestracji i zalogowaniu użytkownik staje się obiektem, który może wykonywać dodatkowe rzeczy. Może on obserwować streamera, uczestniczyć w czacie, a także wysyłać datki. Rejestruje on transakcje i przekazuje informację do modułu portfela twórcy. Użytkownik może również wykupić subskrypcję kanału, który comiesięcznie nalicza mu opłaty.

Użytkownik może być streamerem poprzez aktywację w opcjach kanału. Streamer uruchamia transmisję poprzez ModułStreamowania, który odpowiada za obsługę transmisji wideo. Streamer może czatować z widzami. W trakcie streama ModułCzatu obsługuje wiadomości, a streamer jako właściciel kanału może blokować na zawsze i wyciszać na jakiś czas użytkowników.

Każda donacja, reklama albo subskrypcja przechodzi przez ModułRozliczeń, który zapisuje je na koncie streamera. Streamer może sprawdzić ile ma pieniędzy. Gdy

chce wypłacić pieniądze, wywołuje ModułWypłat. Umożliwia to wybór metody płatności i zapisuje zlecenie w historii. Historia operacji jest zapisana w module ModułTransakcji.

Administrator systemu działa jako obiekt Admin i ma dostęp do modułu ModułZarządzania. Może on przeglądać użytkowników, blokować konta, moderować transmisje oraz zarządzać kategoriami. W przypadku zgłoszeń od użytkowników, ModułWsparcia przekazuje zgłoszenie do administratora, który może je rozpatrzyć i zamknąć.

System zbiera informacje o działaniach użytkowników i streamerów. ModułStatystyk analizuje informacje o oglądalności, aktywności, przychodach i tworzy uproszczone raporty publiczne i dla administratora.

SlopStream operuje na trzydziestu serwerowniach w dwudziestu pięciu krajach. Wszystkie serwerownie łączą się ze sobą za pomocą sieci Internet. Każda serwerownia jest wynajęta w wyspecjalizowanej przestrzeni. Do internetu kablem Gigabit Ethernet podłączono przełącznik, który podpięto do 5 serwerów obliczeniowych, gdzie na każdy serwer przypada 128 GB RAMu i Intel Core i9 3.2GHz. Do tych serwerów przydziela się użytkowników serwisu poprzez komponent serwera-koordynatora: "LoadBalancer". Podpięto też 5 serwerów bazodanowych (16GB RAMU na każdy) do tego samego przełącznika. Wszystkie 10 serwerów jest podpięte kablem Gigabit Ethernet. Każdy serwer główny ma komponent Logger i StreamDetector, który implementuje interfejs StreamDetection.

Aby zapewnić niezawodność w działaniu serwisu, bazy danych są oddzielone, ale są zsynchronizowane. W każdej serwerowni jeden z serwerów koordynuje synchronizacją baz danych co jakiś czas. Każdy serwer bazodanowy ma bazę danych BazaGłówna. Każda BazaGłówna implementuje interfejsy IRead i IWrite. Logi są zapisywane w dedykowanej bazie danych przypisanej do danej serwerowni.

Użytkownicy mogą pobrać aplikację mobilną SlopStream Mobile, aby dostawać powiadomienia o rozpoczętych transmisjach przez twórców treści, których obserwują. Aplikacja korzysta z interfejsu StreamDetection.

Każdy slopstreamer, który przekroczy próg 50 000 obserwujących, może aplikować o Kartę Slopstreamera. Proces jej wytwarzania jest następujący: użytkownik wnioskuje o wyrobienie Karty - system weryfikuje, czy przekroczył wymagany próg obserwacji. Następnie, użytkownik jest proszony o potwierdzenie tożsamości - można to wykonać na dwa sposoby: użytkownik pokazuje dowód osobisty na połączeniu z obsługą albo potwierdzenie przez przelew 1,00 PLN na konto (użytkownik musi przystać PDF z potwierdzenia płatności, który to PDF wymaga późniejszego potwierdzenia przez obsługę). W trakcie gdy potwierdzenie tożsamości jest weryfikowane (może to zająć do 3 dni roboczych) użytkownik może dostosować wygląd zweryfikowanego kanału i karty.

Następnie system oczekuje, aż użytkownik wprowadzi adres dostawy Karty - i tam karta jest dostarczana.

Użytkownik może zapisywać zapisane historyczne transmisje na żywo w tzw. playlistach. Playlisty są trzymane w bazie danych jako struktura hierarchiczna rodzic-dziecko. Każda playlista ma nazwę, UUID właściciela, typ dostępu (prywatny, publiczny) i elementy potomne. Playlista, oprócz samych filmów też może być elementem potomnym co umożliwia zapisywanie folderów filmów w folderach filmów. Użytkownik może zmienić typ dostępu poprzez interfejs oraz usunąć playlistę.

Każda zakończona transmisja zapisywana jest w serwerowni, do której przypisany jest streamer w czasie zakończenia transmisji, a następnie przekazywana do komponentu Synchronizator, który kopiuje tę transmisję pomiędzy inne serwery i serwerownie.

2. Cel budowania systemu, jego zakres oraz kontekst, przewidywalne mierzalne i niemierzalne korzyści z jego wdrożenia:

Platforma SlopStream to serwis streamingowy łączący widzów z ich ulubionymi twórcami z całego świata z różnorodnych dziedzin, takich jak gaming, sport czy lifestyle.

- **Cel budowania systemu:**

Głównym celem jest dostarczenie wydajnej, skalowalnej platformy umożliwiającej kontakt twórców z widzami w czasie rzeczywistym, czy ułatwić wejście w karierę streamingu.

- **Zakres systemu:**

- Witryna z funkcjonalnym GUI
- Aplikacja mobilna
- Panel użytkownika, twórcy oraz administratora
- Chat umożliwiający interakcje w czasie rzeczywistym oraz chat prywatny
- Moduł odtwarzania w czasie rzeczywistym (LIVE)

- System VOD (powtórki, archiwa)
- Moduł finansowy (obsługa donacji, subskrypcji, wirtualnego portfela twórcy)
- System administracyjny (weryfikacja, nakładanie blokad, moderacja zgłoszeń)
- Moduł statystyk (zbieranie statystyk i generowanie raportów)
- Rozproszona infrastruktura serwerowa (synchronizacja rozproszonych baz danych, obsługa LoadBalancera)

• Kontekst Systemu:

System SlopStream funkcjonuje w globalnym, rozproszonym środowisku sieciowym (25 krajów, 30 serwerowni). Wchodzi on w interakcje z następującymi elementami:

- Użytkownicy końcowi: widzowie (przeglądający treści) oraz streamerzy (tworzący treści)
- Zewnętrzne systemy płatności: niezbędne do obsługi donacji, subskrypcji
- Serwery e-mail: wykorzystywane do wysyłania linków weryfikacyjnych i powiadomień
- Fizyczna logistyka: Proces wysyłki karty SlopStreamera wymaga interakcji z firmami kurierskimi

• Przewidywalne mierzalne korzyści z wdrożenia

1. Przychody z prowizji od każdej transakcji (donacje, subskrypcje):

Miesięczny przychód platformy zależy od liczby aktywnych subskrypcji, ilości zrealizowanych donacji oraz wyświetlonych reklam. Platforma pobiera stałą prowizję 25% od każdej takiej transakcji.

Miesięczny przychód obliczany jest następującym wzorem:

$$\text{Przychód} = 25\% * (\text{sub} + \text{don} + \text{ad})$$

Sub – suma wszystkich opłat za subskrypcję (20 PLN) w miesiącu (ilość_subskrybcji * 20))

Don – suma wszystkich donacji wysłanych do streamerów

Ad – suma przychodów z reklam (1 wyświetlenie = 0,15 PLN, ilość * 0,15 PLN)

Przy założeniu 100 000 aktywnych subskrybcji (20 PLN), 500 000 donacji (średnio 10 PLN) oraz 15 000 000 wyświetlonych reklam (0,15 za każdą reklamę) przewidywany miesięczny przychód systemu wynosi 2 312 500,00 PLN

2. Wysoka dostępność usług:

Dzięki rozproszeniu na 30 serwerowni w 25 krajach, mechanizmu LoadBalancer oraz rozproszeniu baz danych, system znacznie zredukuje czas niedostępności usług. Przewidywany czas niedostępności usługi to 2 godziny w skali roku.

Sposób wyliczenia:

$$\text{Roczny_czas_niedostępności} = \text{liczba_awarii_w_roku} * \text{śr_czas_rozwiązania}$$

W przypadku awarii pojedynczego serwera, komponent LoadBalancer automatycznie przekieruje ruch do innych aktywnych węzłów co sprawi, że w najgorszym przypadku użytkownik będzie musiał poczekać na ponowne załadowanie się storny. Rozproszone bazy danych zapewniają ciągłość dostępu do danych nawet przy lokalnych usterkach infrastruktury.

Nie możemy jednak założyć stu procentowej niezawodności systemu, dlatego założony został 2 godzinny okres niedostępności.

- **Przewidywalne niemierzalne korzyści z wdrożenia:**

1. Budowa zaangażowania i lojalności społeczności:

Implementacji chatu na żywo oraz prywatnych wiadomości pozwala użytkownikom na integrację z twórcami oraz innymi widzami. Zapsy transmisji oraz tworzenie playlist sprawi, że użytkownicy chętniej będą wracać na platformę w celu ponownego obejrzenia transmisji. Dzięki temu budowane jest trwałe przywiązanie użytkownika do platformy.

2. Poczucie bezpieczeństwa użytkowników i twórców:

Dzięki 30 dniowemu okresowi wygaśnięcia konta oraz rocznego przechowywania danych przed usunięciem użytkownik jak i twórca ma czas na zmianę decyzji oraz ewentualną próbę odzyskania danych. Rozproszenie baz danych zmniejsza ryzyko utraty danych przez awarie.

3. Poczucie niezawodności systemu:

Rozproszona infrastruktura serwerów sprawi, że użytkownicy będą łączeni z najbardziej dostępnymi węzłami. Zmniejszy to znacznie opóźnienia co jest istotne dla wrażenia interaktywności podczas transmisji na żywo.

3. Słownik (definiujący ważne, specyficzne terminy związane z dziedziną problemu, wykorzystywane w projekcie)

- Stream – Transmisja wideo prowadzona w czasie rzeczywistym.
- Streamer – Osoba prowadząca transmisję na żywo
- Moderator/Support – Rola personelu administracyjnego odpowiedzialna za weryfikację treści pod kątem etycznym oraz obsługę zgłoszeń (ticketów) użytkowników.
- VOD (Video on Demand) – Zapis Streama dostępny do obejrzenia w dowolnym momencie po jego zakończeniu.
- Donacja – Dobrowolna wpłata finansowa przekazywana przez widza dla streamera.

- LoadBalancer – Komponent programowo - sprzętowy odpowiedzialny za rozdzielanie ruchu sieciowego od użytkowników pomiędzy 150 dostępnych serwerów aplikacji w celu zapobiegania przeciążeniom
- StreamDetector – Autorski moduł systemowy działający w warstwie obliczeniowej którego zadaniem jest natychmiastowe wykrywanie rozpoczęcia transmisji przez streamera i powiadamianie bazy danych oraz aplikacji mobilnej.
- Synchronizator – Komponent odpowiedzialny za replikację i przesyłanie danych wideo oraz informacji o stanie systemu pomiędzy 30 fizycznymi serwerowniami w celu zapewnienia spójności danych.
- Logger – Moduł systemowy służący do ciągłego zbierania, przetwarzania i zapisywania logów (zdarzeń systemowych i błędów) do odseparowanej bazy danych logów.

4. Perspektywa przypadków użycia

4.1 Diagramy przypadków użycia:



4.1.1. Opisy tekstowe wszystkich aktorów:

- **Gość:** Osoba odwiedzająca serwis bez zalogowania. Może przeglądać transmisje na żywo oraz playlisty zapisane przez innych użytkowników. Transmisje i playlisty może filtrować i sortować. Gość ma możliwość rejestracji i logowania.
- **Użytkownik:** Posiada założone konto w serwisie oraz jest zalogowany. dziedziczy uprawnienia gościa. Może dołączać do chatu, obserwować twórców, wysyłać donacje, subskrybować twórców, tworzyć i zarządzać playlistami oraz zarządzać własnym kontem.
- **Twórca:** Zalogowany użytkownik, który aktywował opcję bycia twórcą. Dziedziczy uprawnienia użytkownika. Posiada uprawnienia do prowadzenia transmisji na żywo, moderacje swojego chatu a także sprawdzania stanu zarobków i wypłacania zgromadzonych środków.
- **Administrator:** Pracownik platformy zarządzający jej funkcjonowaniem. Dziedziczy uprawnienia użytkownika. Ma dostęp do modułu zarządzania, gdzie może modyfikować kategorie, blokować konta, moderować transmisje, oraz weryfikować kanały twórców.

4.1.2. Opisy tekstowe wybranych przypadków użycia:

Opis przypadku użycia „Zarejestruj się”:

1. Uczestniczący aktorzy:
 - Gość
2. Podstawowy ciąg zdarzeń:
 - Gość wybiera opcje rejestracji
 - Wypełnia formularz podając nazwę użytkownika, email i hasło
 - System wysyła dane do serwera, weryfikuje poprawność danych
 - System zapisuje użytkownika w bazie
 - Gość otrzymuje komunikat o sukcesie rejestracji

3. Alternatywne ciągi zdarzeń:

- a) System stwierdza niekompletność lub niepoprawność danych – System wyświetla formularz ponownie wskazując pola zawierające błędy. Klient poprawia dane i ponownie wysyła formularz

- b) Gość rezygnuje z rejestracji - Gość przerywa proces rejestracji.

System przerywa proces i wraca do stanu przed rozpoczęciem rejestracji

4. Zależności czasowe:

- a) Częstotliwość wykonania: duża (ciągły napływ nowych użytkowników)
- b) Przewidywane spiętrzenia: Zwiększona liczba nowych użytkowników spowodowana popularnym wydarzeniem na platformie
- c) Typowy czas realizacji: 1 minuta
- d) Maksymalny czas realizacji: nieokreślony

5. Wartości uzyskiwane przez aktora po zakończeniu przypadku użycia:

- Gość uzyskuje dostęp do konta na platformie.
- Gość staje się użytkownikiem.

Opis przypadku użycia „Wyślij donację dla twórcy”:

1. Uczestniczący aktorzy:

- Użytkownik

2. Podstawowy ciąg zdarzeń:

- Użytkownik w trakcie trwania transmisji wybiera opcje donacji
- podaje kwotę oraz ewentualną wiadomość tekstową
- System weryfikuje płatność
- Rejestracja transakcji - środki trafiają do wirtualnego portfela twórcy
- Donacja użytkownika z wiadomością wyświetla się na chacie i transmisji twórcy

3. Alternatywne ciągi zdarzeń:

- a) System odrzuca płatność - Bramka płatności odrzuca transakcję (np. odrzucenie karty płatniczej). System przerywa operację, nie rejestruje transakcji oraz wyświetla użytkownikowi komunikat o niepowodzeniu
 - b) Użytkownik rezygnuje z donacji - System przerywa proces i wraca do stanu sprzed rozpoczęcia operacji
4. Zależności czasowe:
- a) Częstotliwość wykonania: zależna od ilości użytkowników uczestniczących w transmisjach, średnio kilkadziesiąt razy dziennie
 - b) Przewidywane spiętrzenia: Godziny wieczorne, weekendy. Transmisja popularnego twórcy
 - c) Typowy czas realizacji: 30 sekund
 - d) Maksymalny czas realizacji: nieokreślony
5. Wartości uzyskiwane przez aktora po zakończeniu przypadku użycia:
- Potwierdzenie przyjęcia płatności.
 - Informacja o donacji na chacie transmisji.

Opis przypadku użycia: „Rozpocznij transmisję”:

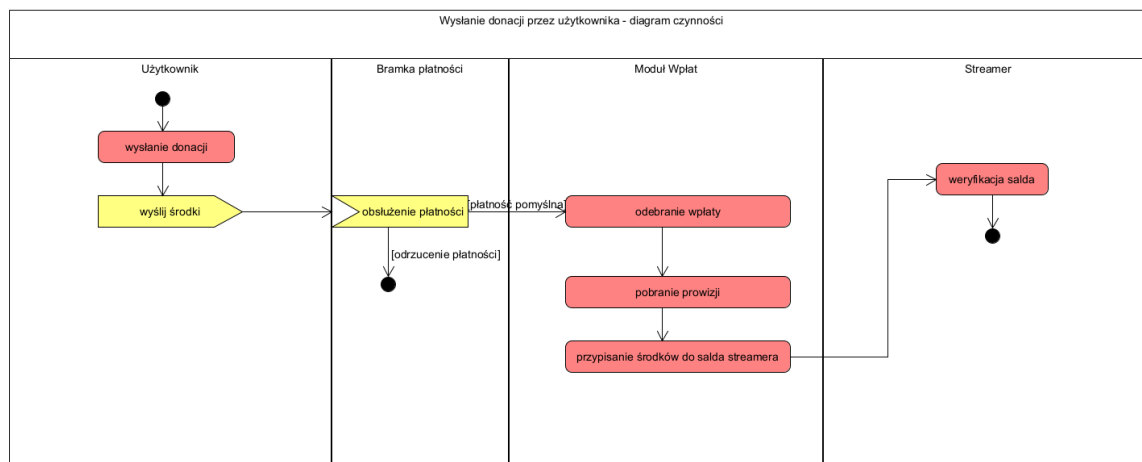
1. Uczestniczący aktorzy:
 - Twórca
2. Podstawowy ciąg zdarzeń:
 - Twórca w panelu twórcy wybiera opcję rozpoczęcia transmisji.
 - System inicjalizuje połączenie za pomocą Modułu Streamowania.
 - Status twórcy zmienia się na aktywny.
 - Użytkownicy obserwujący twórcę otrzymują powiadomienie o transmisji.
 - Transmisja jest uruchomiona, otwiera się chat transmisji, zbierane są statystyki.
3. Alternatywne ciągi zdarzeń:
 - a) Błąd połączenia - System nie może nawiązać połączenia z twórcą. Twórca otrzymuje komunikat o błędzie, transmisja nie jest rozpoczynana. Powiadomienia nie są wysyłane

- b) Twórca rezygnuje z rozpoczęcia transmisji - System przerywa proces i wraca do stanu przed rozpoczęciem procesu
4. Zależności czasowe:
- a) Częstotliwość wykonania: zależna od liczby aktywnych twórców, średnio kilkadziesiąt razy dziennie
 - b) Przewidywane spiętrzenia: Godziny wieczorne
 - c) Typowy czas realizacji: 15 sekund
 - d) Maksymalny: nieokreślony
5. Wartości uzyskiwane przez aktora po zakończeniu przypadku użycia:
- Twórca pomyślnie uruchamia swoją transmisję umożliwiając interakcję z publicznością, budowanie zasięgów i potencjalne generowanie dochodów z subskrypcji i donacji

4.2. Diagramy czynności:

- Wysłanie donacji przez użytkownika:

Użytkownik ma możliwość wysłania donacji poprzez specjalny panel, określa kwotę oraz metodę płatności a następnie wysyła żądanie do API bramki płatności, gdy transakcja się nie powiedzie API bramki odrzuca transakcje, w innym wypadku Moduł Wpłat odbiera informacje o wpłacie, a następnie przypisuje daną kwotę do konta streamera, po wcześniejszym odjęciu prowizji.



Użytkownik - użytkownik końcowy

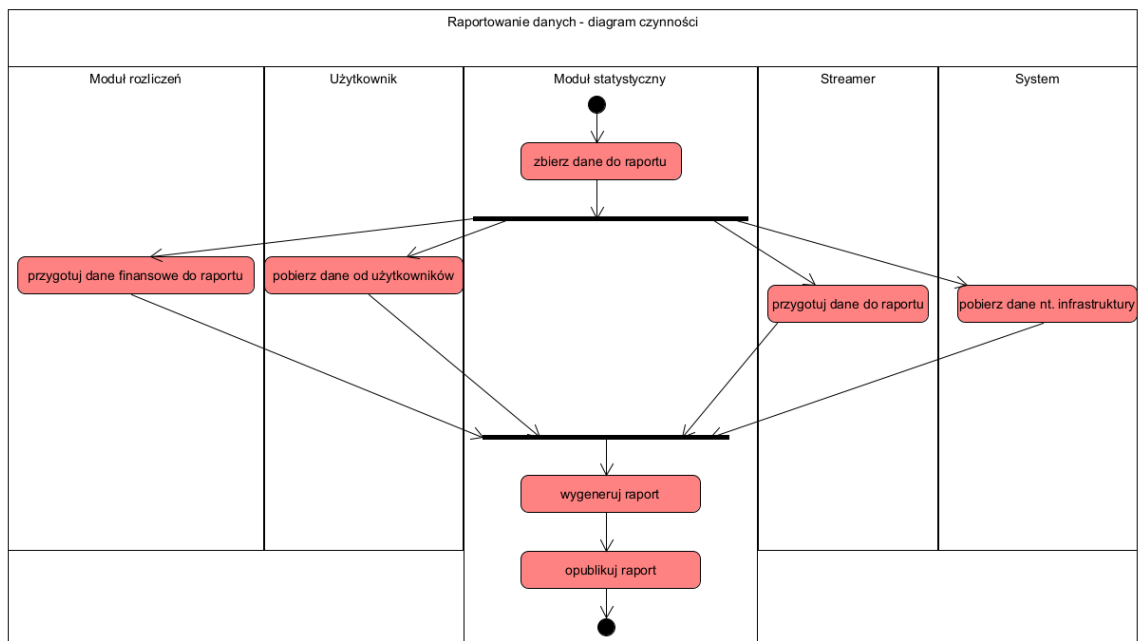
Bramka płatności - zewnętrzne API obsługujące płatności

Moduł Wpłat - moduł zarządzający transakcjami i przypisywaniem salda docelowym streamerom.

Streamer - użytkownik z prawami do transmisji, na którego konto trafiają donacje

- **Raportowanie danych statystycznych:**

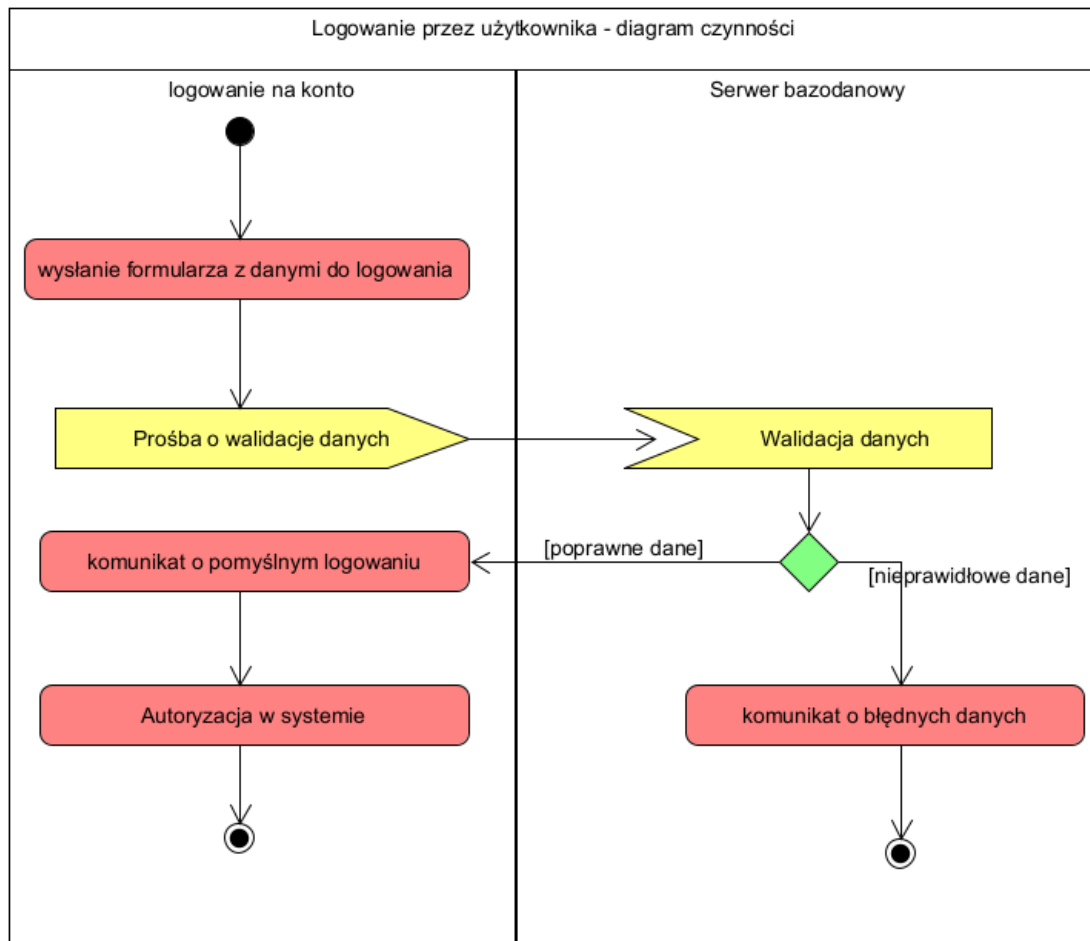
Klasa która obsługuje generowanie raportów statystycznych dla platformy, zbiera dane z poszczególnych klas takich jak: Użytkownik, Moduł rozliczeń, Streamer, System. Na bazie informacji pobranych z klas generuje raport, który następnie jest publikowany w panelu statystycznym administratora.



- Logowanie użytkownika

Do uzyskania autoryzacji w systemie, potrzebne jest logowanie.

Użytkownik do zalogowania potrzebuje swojego adresu e-mail oraz hasła, które następnie wprowadza w formularz. Po wypełnieniu formularza z danymi i zatwierdzeniu - wysyłane jest żądanie na endpoint odpowiedzialny za weryfikację użytkowników. Kiedy dane okażą się poprawne - użytkownik dostaje komunikat o pomyślnym logowaniu, a następnie dostaje autoryzację w systemie. W przeciwnym wypadku użytkownik otrzymuje komunikat o błędnych danych i jest zmuszony do wypełnienia formularza ponownie.

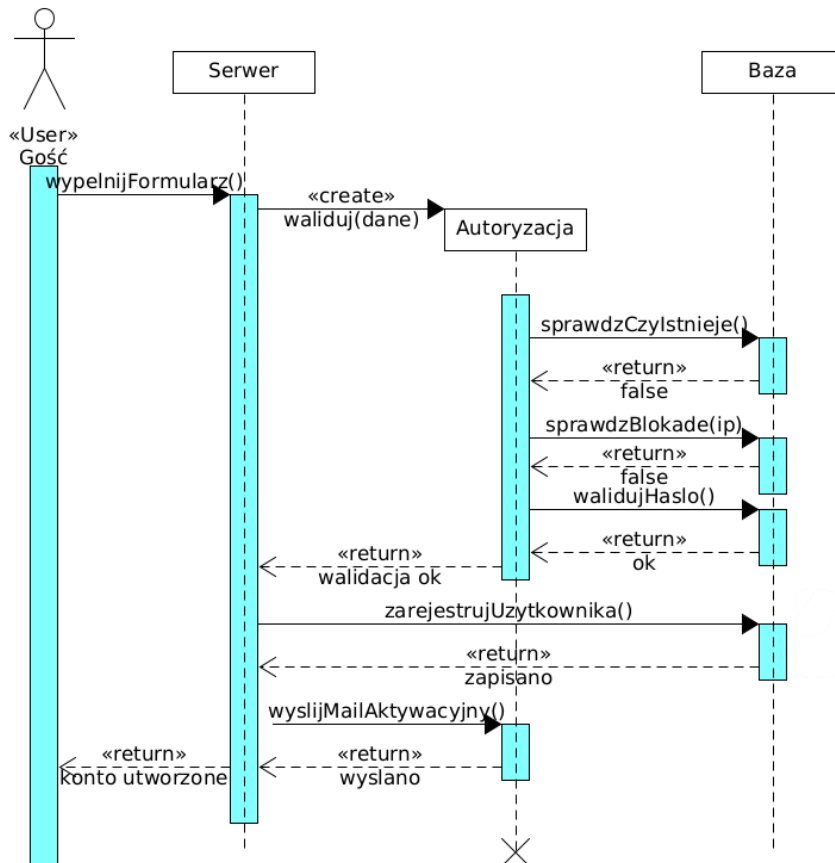


4.3. Diagramy interakcji (przebiegu) z opisem tekstowym komunikatów:

- **Diagram 1: Zarejestruj**

W systemie serwisu streamingowego SlopStream niezalogowany gość chce utworzyć nowe konto. Po wypełnieniu formularza rejestracyjnego, dane trafiają na główny serwer, który tworzy tymczasowy obiekt autoryzacji w celu zwalidowania wprowadzonych informacji. Autoryzacja odpytuje bazę danych, sprawdzając kolejno:

czy użytkownik o takich danych już istnieje, czy jego adres IP nie posiada blokady oraz czy wprowadzone hasło spełnia wymogi bezpieczeństwa. Po pomyślnej walidacji, serwer zleca bazie rejestrację użytkownika, a obiektowi autoryzacji wysłanie e-maila aktywacyjnego



Serwer:

- `WypełnijFormularz()` - odbiera dane z formularza rejestracyjnego.

Baza:

- `SprawdzCzyIstnieje()` – zwraca informację (true/false), czy dany e-mail jest już zajęty.
- `SprawdzBlokade(ip)` - zwraca informację (true/false), czy na IP nałożona jest blokada.

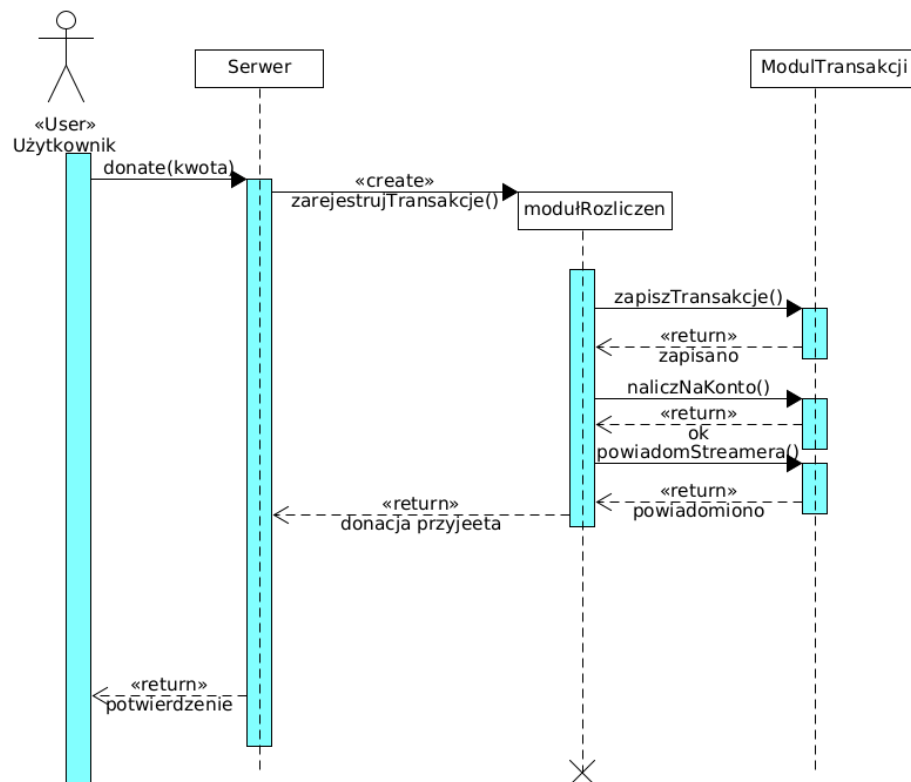
- walidujHaslo() - sprawdza złożoność i siłę hasła (zwraca ok/błąd).
- zarejestrujUżytkownika() - fizycznie zapisuje nowego użytkownika w bazie.

Autoryzacja:

- wyslijMailAktywacyjny() - wysyła wiadomość z linkiem na e-mail gościa.
- waliduj(dane) - konstruktor/metoda weryfikująca poprawność danych.

• Diagram 2: Wyślij donację twórcy

W systemie serwisu streamingowego SlopStream zalogowany użytkownik chce wesprzeć finansowo twórcę. Użytkownik podaje kwotę donacji, a żądanie trafia do głównego serwera. Serwer tworzy tymczasowy obiekt modułu rozliczeń w celu obsłużenia całego procesu płatności. Moduł ten komunikuje się z głównym modulem transakcji, zlecając mu kolejno trzy operacje: fizyczne zapisanie transakcji w historii, naliczenie środków na konto streamera oraz wysłanie powiadomienia do obdarowanego twórcy. Po pomyślnym zrealizowaniu wszystkich kroków, moduł rozliczeń zwraca informację o przyjęciu donacji do serwera, po czym jest usuwany z pamięci. Na koniec serwer wyświetla użytkownikowi potwierdzenie operacji.



Serwer:

- donate(kwota) – odbiera żądanie wysłania donacji wraz z określoną kwotą od użytkownika.

modulRozliczen:

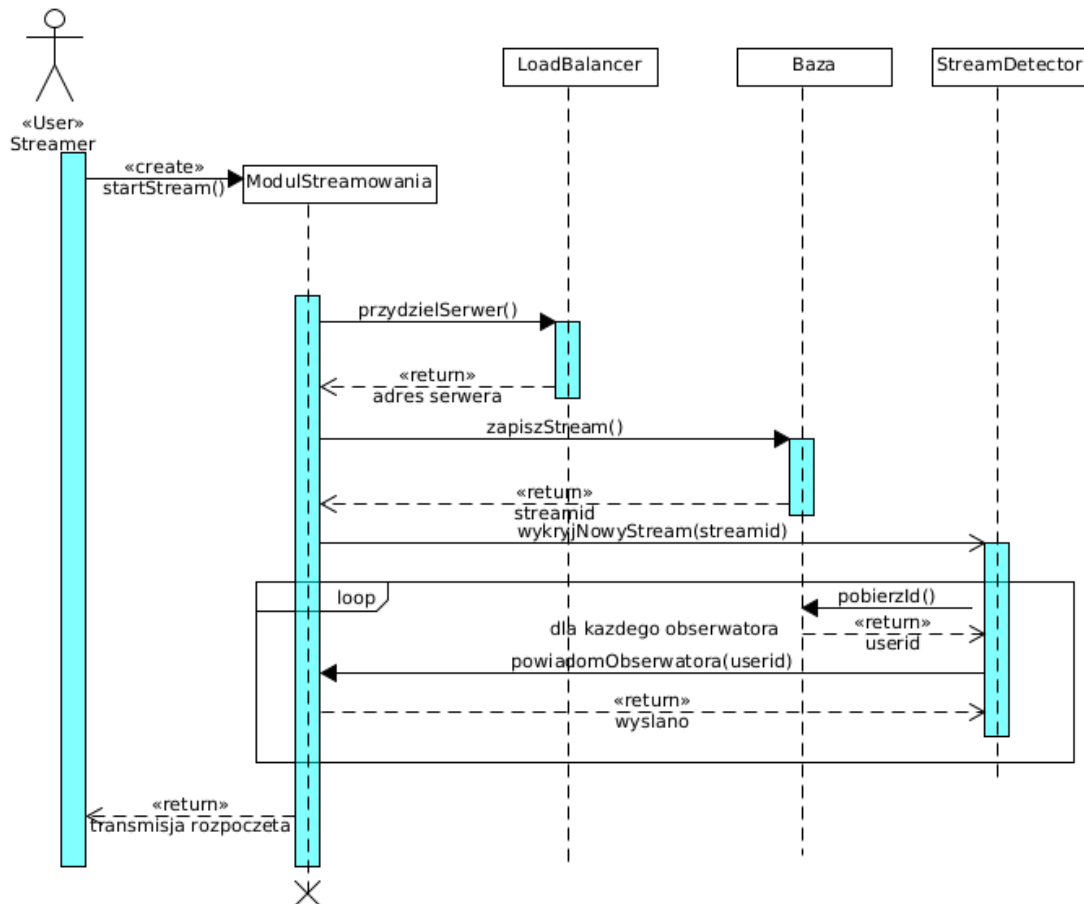
- zarejestrujTransakcje() – metoda tworząca tymczasowy obiekt modułu rozliczeń.

mdulTransakcji:

- zapiszTransakcje() – trwale rejestruje dane o nowej wpłacie (zwraca komunikat "zapisano").
- naliczNaKonto() – przypisuje przebrane środki do salda twórcy (zwraca status "ok").
- powiadomStreamera() – generuje powiadomienie o nowym datku dla twórcy (zwraca "powiadomiono").

- **Diagram 3: Rozpocznij transmisje**

W systemie serwisu streamingowego SlopStream twórca (Streamer) chce rozpocząć transmisję na żywo. Inicjuje on proces, wywołując metodę tworzącą obiekt modułu streamowania. Moduł ten w pierwszej kolejności komunikuje się z LoadBalancerem w celu przydzielenia odpowiedniego serwera. Po otrzymaniu adresu serwera, moduł zgłasza do bazy danych żądanie zapisania nowego streama, w zamian otrzymując jego unikalny identyfikator (streamid). Następnie moduł streamowania informuje komponent StreamDetector o nowej transmisji. StreamDetector rozpoczyna działanie w pętli dla każdego obserwatora twórcy: pobiera jego identyfikator (userid) z bazy, a następnie zleca modułowi streamowania wysłanie powiadomienia do tego użytkownika. Po zakończeniu rozsyłania powiadomień do wszystkich obserwatorów, moduł streamowania zwraca twórcy komunikat o pomyślnym rozpoczęciu transmisji



ModulStreamowania :

- startStream() – metoda inicjująca proces rozpoczęcia transmisji.
- powiadomObserwatora(userid) – wysła powiadomienie do widza o konkretnym identyfikatorze (zwraca komunikat "wyslano").

LoadBalancer:

- przydzielSerwer() – przypisuje zasoby do obsługi strumienia i zwraca adres serwera.

Baza:

- zapiszStream() – rejestruje nową transmisję w systemie i zwraca jej streamid.
- pobierzId() – wewnątrz pętli pobiera z bazy danych userid kolejnego obserwatora.

StreamDetector:

- `wykryjNowyStream(streamid)` – przyjmuje informację o starcie transmisji i rozpoczyna proces powiadamiania.

5. Perspektywa projektowa:

5.0. Proponowana architektura systemu:

Serwis SlopStream opiera się o architekturę wielowarstwową wykorzystującą wyspecjalizowane moduły biznesowe odpowiadające za poszczególne funkcje.

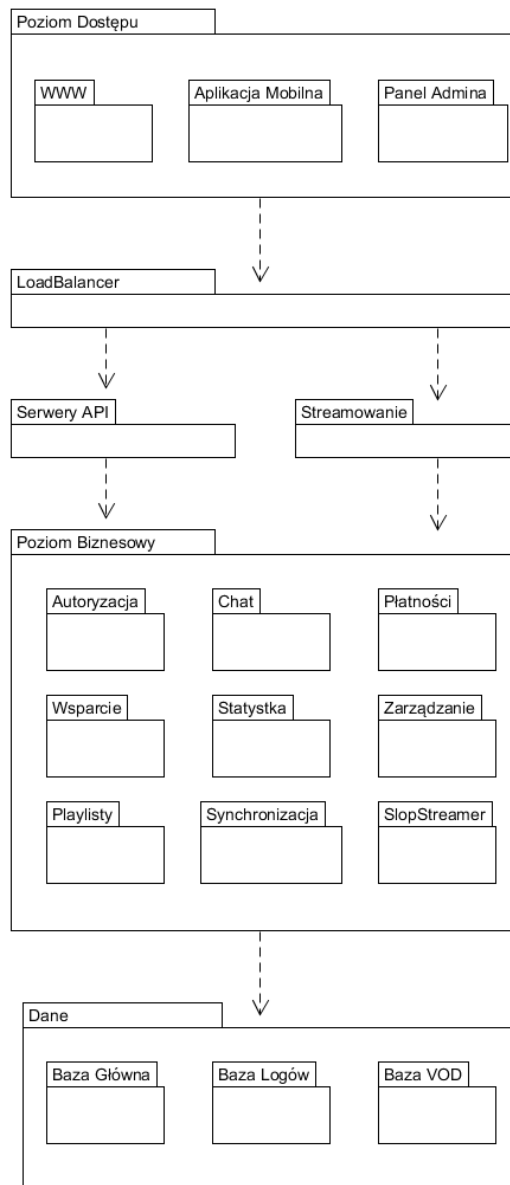
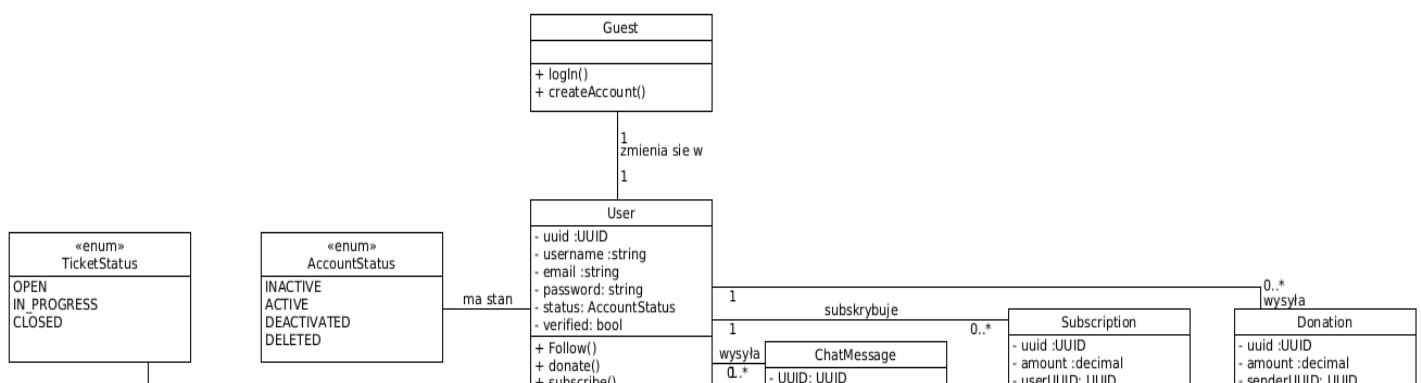


Diagram pakietów przedstawia warstwy wraz z modułami znajdującymi się w nich.

5.1. Diagramy klas:



5.2. Uporządkowany alfabetycznie wykaz wszystkich klas:

1. Admin:

Klasa potomna klasy User, jest to reprezentacja użytkownika o prawach do zarządzania serwisem.

Atrybuty:

-assignedTicket : List<UUID> - Lista zawierająca id przypisane do tego admina.

Metody:

+banUser(user : User) : void – Zbanowanie użytkownika.

+ManageCategories() : void - Zarządzanie kategoriami.

+resolveTicket(ticket : Ticket) : void - Rozwiązaywanie zgłoszeń.

2. AccountStatus

Enum zawierający statusy jakie może przyjmować konto.

INACTIVE – Konto nie aktywne.

ACTIVE – Konto aktywne.

DEACTIVATED – Konto zdezaktywowane.

DELETED – Konto usunięte.

3. Category:

Reprezentacja kategorii jaką może być opisany stream.

Atrybuty:

- uuid : UUID – Id kategorii.

- name : String – Nazwa kategorii.

4. ChatMessage:

Wiadomość wysłana poprzez chat.

Atrybuty:

- uuid : UUID – Id wiadomości.

- senderUUID : UUID – Id użytkownika wystającego wiadomość.

- content : String – Zawartość wiadomości.

- sendDate : DateTime – Data wysłania wiadomości.

5. Donation:

Klasa datku reprezentująca pieniądze przestana od widza do streamera.

Atrybuty:

- uuid : UUID – Id datku.

- amount : Decimal - Przestana kwota.

- senderUUID : UUID – Id wystającego.

- receiverUUID : UUID – Id przyjmującego.

- createdAt : DateTime – Data dokonanie transakcji.

6. Guest:

Klasa reprezentująca niezalogowanego użytkownika serwisu.

Metody:

+ login(login : String, password : String) - Metoda umożliwiająca zalogowanie się do serwisu.

+ createAccount(email : String, login : String, password : String) - Metoda umożliwiająca utworzenie konta w serwisie.

7. PlaylistComponent:

Klasa abstrakcyjna wymaga do osiągnięcia wzorca projektowego composite, który realizowany jest przez playliste.

Metody:

+ display() : void – Metoda wyświetlenie elementu.

8. PlaylistComposite:

Klasa reprezentująca playliste oraz umożliwiająca zarządzanie nią.

Atrybuty:

- uuid : UUID – Id playlisty.

- name : String – Nazwa playlisty.

- ownerUUID : UUID – Id twórcy playlisty.

- accessType : AccessType – Typ dostępu do playlisty.

- children : List< PlaylistComponent> - Lista elementów należących do playlisty.

Metody:

+ add(component : PlaylistComponent) : void – Metoda dodwania elementów do playlisty.

+ remove(component : PlaylistComponent) : void - Metoda usuwania elementów z playlisty.

+ display() : void - Metoda wyświetlenie elementów.

9. PlaylistLeaf:

Klasa reprezentująca pojedynczy stream w playliste.

Atrybuty:

- vodUUID : UUID – Id VOD, który jest elementem playlisty.

Metody:

- display() : void- Metoda wyświetlenie elementu.

10. Stream:

Klasa reprezentująca transmisję na żywo.

Atrybuty:

- streamId : UUID – Id transmisji na żywo.
- title : String - Tytuł transmisji na żywo.
- categories : List<Category> - Lista kategorii.
- viewers : List<User> - Lista widzów.
- startTime : DateTime – Czas rozpoczęcia.
- endTime : DateTime – Czas zakończenia.
- isLive : bool – Czy stream jest aktywny.

Metody:

- + start() : void – Metoda rozpoczynająca transmisję.
- + stop() : void – Metoda kończąca transmisję.

11. StreamerCard:

Karta streamera dająca mu dostęp do benefitów.

Atrybuty:

- uuid: UUID – Id karty.
- ownerUUID : UUID – Id właściciela karty.
- deliveryAddress : String – Adres pocztowy właściciela karty.
- verified : bool – Status karty.

12. Streamer:

Klasa potomna klasy User, jest to reprezentacja użytkownika, który może prowadzić transmisję na żywo.

Atrybuty:

- followers : List<User> - Lista użytkowników obserwujących streamera.
- subscribers : List<User> - Lista użytkowników subskrybujących streamera.

Metody:

- + startStream() - Metoda rozpoczynająca transmisję.
- + stopStream() - Metoda kończąca transmisję.
- + muteUser(user : User, duration : int) : void – Metoda wyciszająca widza, pozbawia go możliwości komentowania.
- + banUser(user : User) : void- Metoda blokująca widza, pozbawia go możliwości interakcji ze streamerem (chat, dostęp do profilu, dostęp do streamów).
- + requestCard() : void – Metoda wnioskująca o wydanie karty streamera.
- + withdraw() : void – Metoda pozwalająca na wypłacenie środków zebranych podczas działalności na platformie.

13. Subscription:

Klasa przechowująca informacje o subskrypcjach użytkowników do streamerów.

Atrybuty:

- uuid : UUID – Id subskrypcji.
- amount : Decimal – Kwota subskrypcji.
- userUUID : UUID – Id subskrybenta.
- streamerUUID : UUID – Id subskrybowanego.
- billingDate : DateTime – Data odnowienia subskrypcji.

14. Ticket:

Klasa przechowująca informacje o zgłoszonych problemach.

Atrybuty:

- uuid : UUID – Id zgłoszenia.
- reporterUUID : UUID – Id zgłaszającego.
- assignedAdminUUID : UUID – Id admina przypisanego do rozwiązania zgłoszenia.
- description : String - Opis zgłoszenia.
- status : TicketStatus – Status w jakim znajduje się zgłoszenie.
- createdAt : DateTime – Data utworzenia.
- resolvedAt : DateTime – Data rozwiązania problemu.

15. TicketStatus:

Enum zawierający statusy jakie może przyjmować zgłoszenie.

OPEN - Zgłoszenie utworzone.

IN_PROGRESS – Zgłoszenie w trakcie rozpatrywania.

CLOSED - Zgłoszenie zostało rozpatrzone/ rozwiązane.

16. User:

Klasa reprezentująca zalogowanego użytkownika serwisu.

Atrybuty:

- uuid : UUID – Id użytkownika.
- username : String – Nazwa użytkownika.
- email : String – Adres poczty internetowej użytkownika.
- password : String - Hasło użytkownika.
- status : AccountStatus – Status konta.
- verified : bool – Informacja o tym czy konto jest zweryfikowane.

Metody:

- + follow(streamer : Streamer) : void – Metoda pozwalająca na obserwowanie streamerów.
- + donate(amount : Decimal) : void - Metoda pozwalająca na przekazywanie datków dla streamerów.
- + subscribe(streamer : Streamer) : void - Metoda pozwalająca na subskrybowanie streamerów.
- + deleteAccount() : void – Metoda pozwalająca na usunięcie konta.

17. Vod:

Zapis z transmisji na żywo w formie video dostępny do obejrzenia w dowolnym momencie.

Atrybuty:

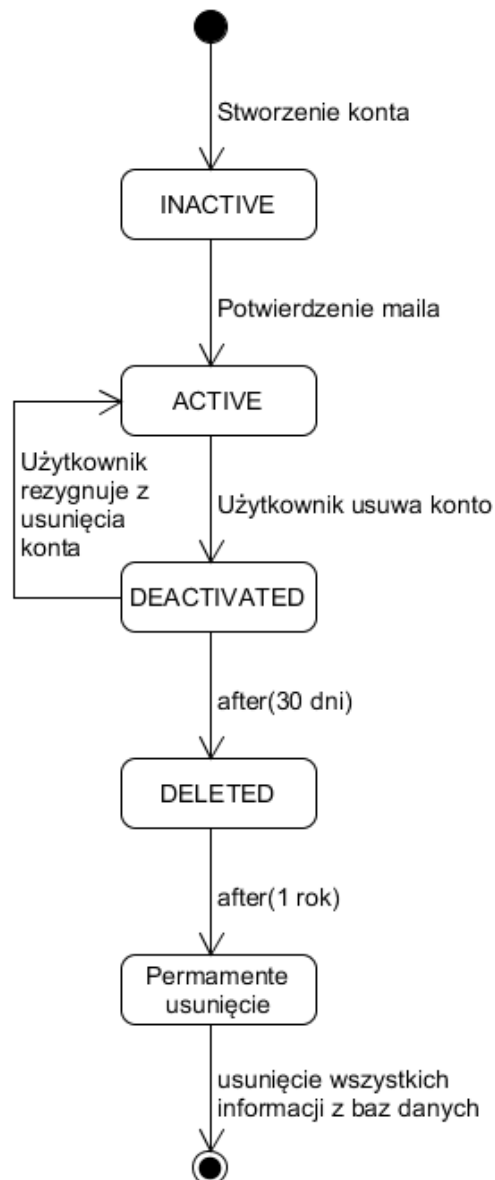
- VodUUID : UUID – Id vodu.
- title : String - Tytuł vodu.
- duration : int – Czas trwania zapisu.
- uploadDate : DateTime – Data wstawienia zapisu na platformę.

Metody:

+ display() : void – Metoda wyświetlająca zapis z transmijsi.

5.3. Diagramy stanów:

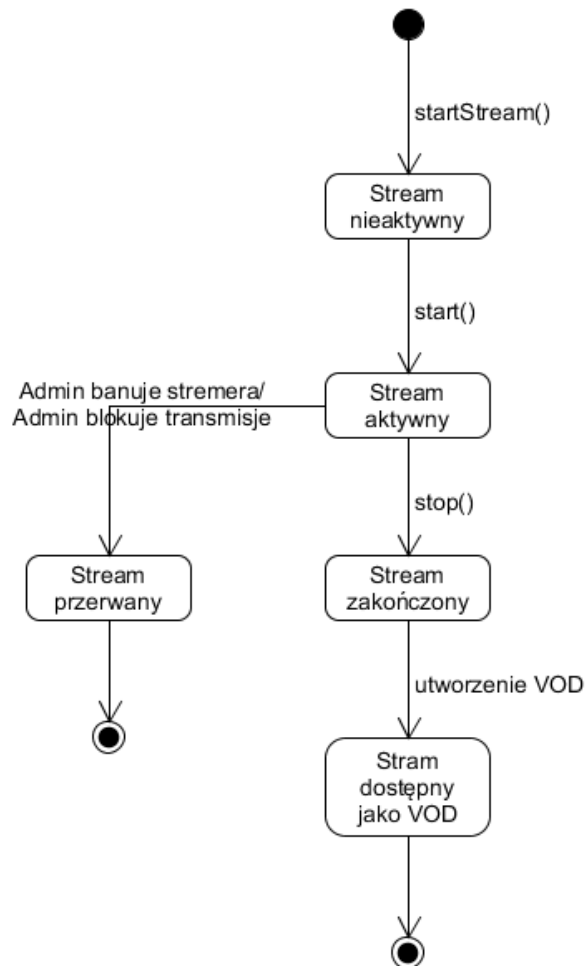
- Diagram 1: Stany konta użytkownika



Użytkownik tworzy konto, które przechodzi w stan INACTIVE. Konto będzie w tym stanie aż do potwierdzenia utworzenia konta przez użytkownika przy użyciu maila po czym zmieni swój stan na ACTIVE dając użytkownikowi pełen dostęp do platformy. Użytkownik może zdecydować o usunięciu konta, przechodzi ono wtedy w stan DEACTIVATED. Użytkownik przez 30 dni ma możliwość odwołania się od decyzji o usunięciu konta co sprawi iż konto wróci do stanu ACTIVE lecz jeżeli tego nie zrobi po upływie tego czasu konto przechodzi w stan DELETED i nie jest już dostępne dla

użytkownika. Po roku od zmiany stanu konta na DELETED wszystkie rekordy związane z tym kontem są usuwane z baz danych.

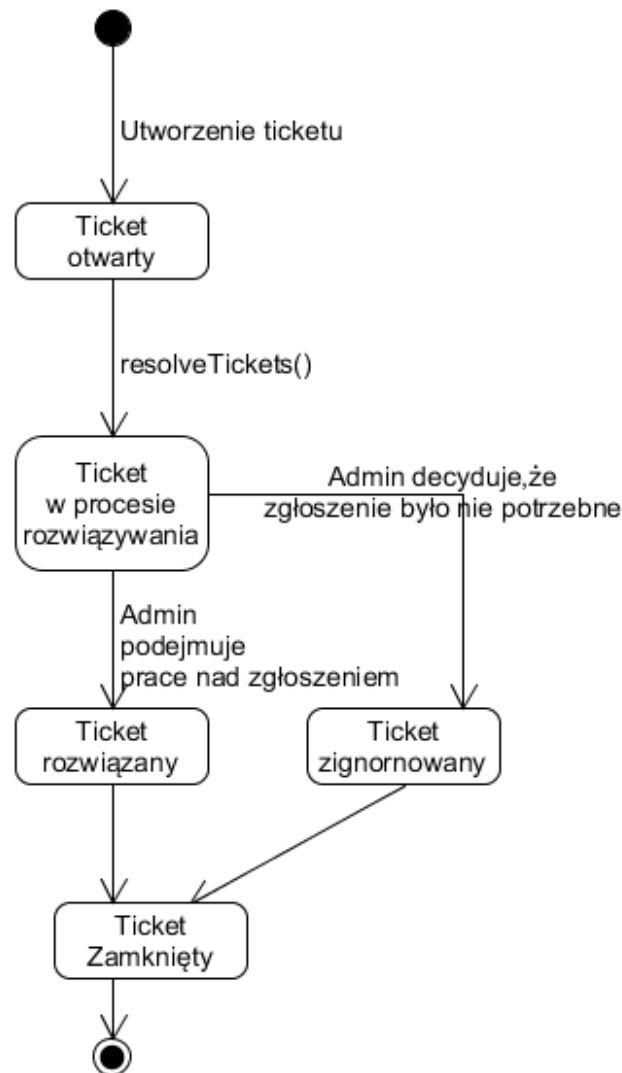
- **Diagram 2: Stany stream**



Użytkownik, który jest streamerem tworzy Streama wywołując metodę `startStream()`. Stream zostaje utworzony i przechodzi w stan nieaktywny w trakcie, którego może zostać ustalony tytuł, kategorie czy inne techniczne parametry transmisji. Stream rozpoczyna się i przechodzi w stan aktywny po wywołaniu metody `start()`. W trakcie transmisji admin może zdecydować o jej przerwaniu bądź o zbanowaniu Streamera co powoduje natychmiastowe zakończenie streama oraz nie utworzenie VOD, stream przechodzi w stan przzerwany. Streamer kończy transmisję przy wykorzystaniu metody `stop()`, stream przechodzi w stan zakończony. Po zakończeniu

transmisji tworzone jest VOD po czym przesyłane jest ono na serwery a stream przechodzi w stan bycia dostępnym jako VOD.

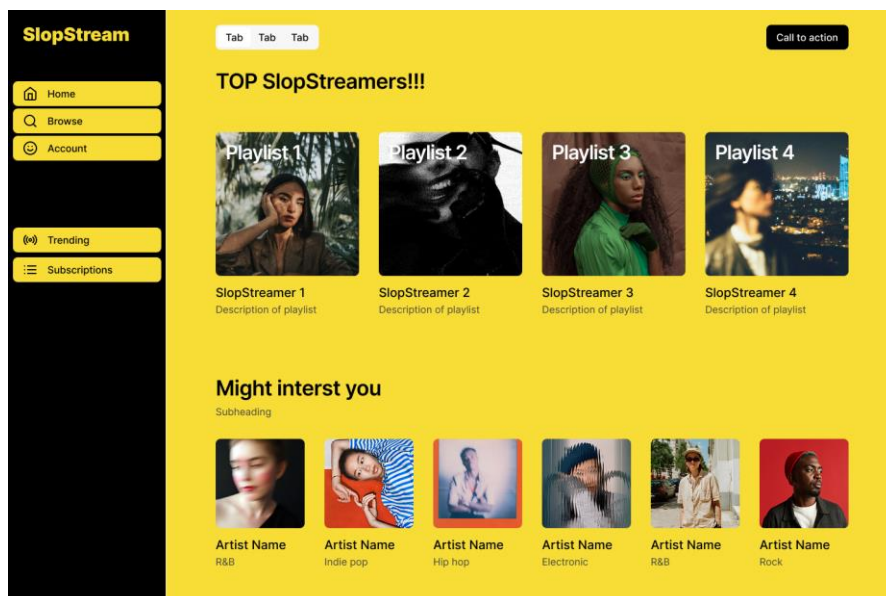
- **Diagram 3: Stany zgłoszenia**



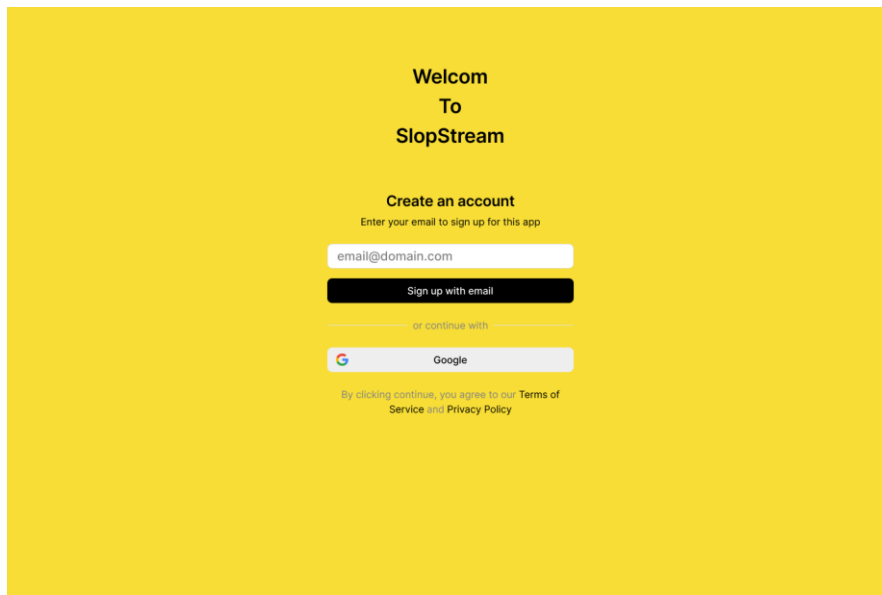
Użytkownik tworzy zgłoszenie, ticket przechodzi w stan otwarty i czeka aż administrator go rozwiąże. Admin wywołuje metodę `reolveTickets()`, która zmienia stan zgłoszenia i informuje, że ticketem już ktoś się zajmuje. W trakcie rozwiązywania adminstrator może podjąć decyzję czy zgłoszenie było słuszne i należy podjąć działanie czy przeciwnie jest nieistotne. Zależnie od decyzji admina ticket jest rozwiązywany bądź ignorowany po czym zmienia swój stan na zamknięty.

5.4 Propozycje interfejsu użytkownika (okno główne, menu główne i podręczne, kluczowe formatki dialogowe, itp...):

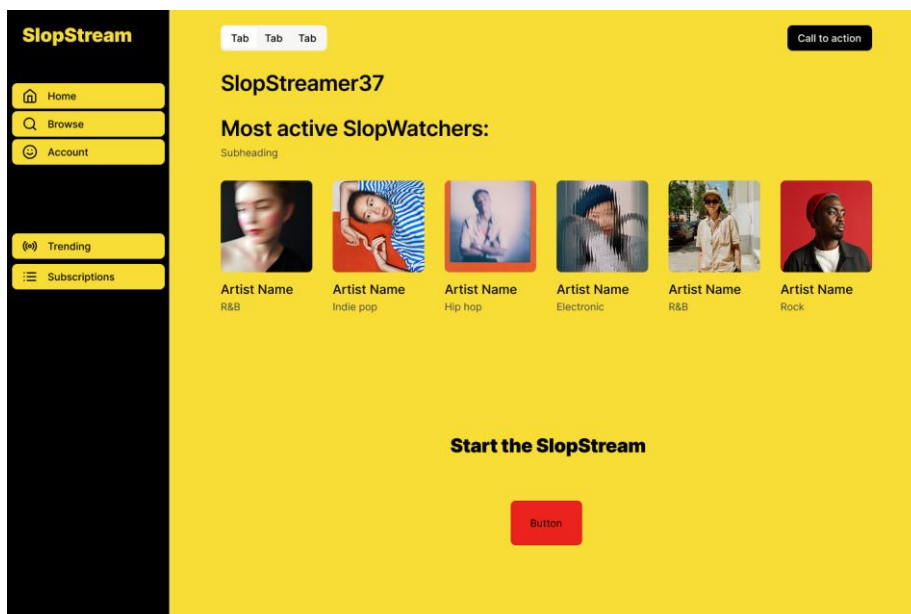
- Strona główna:



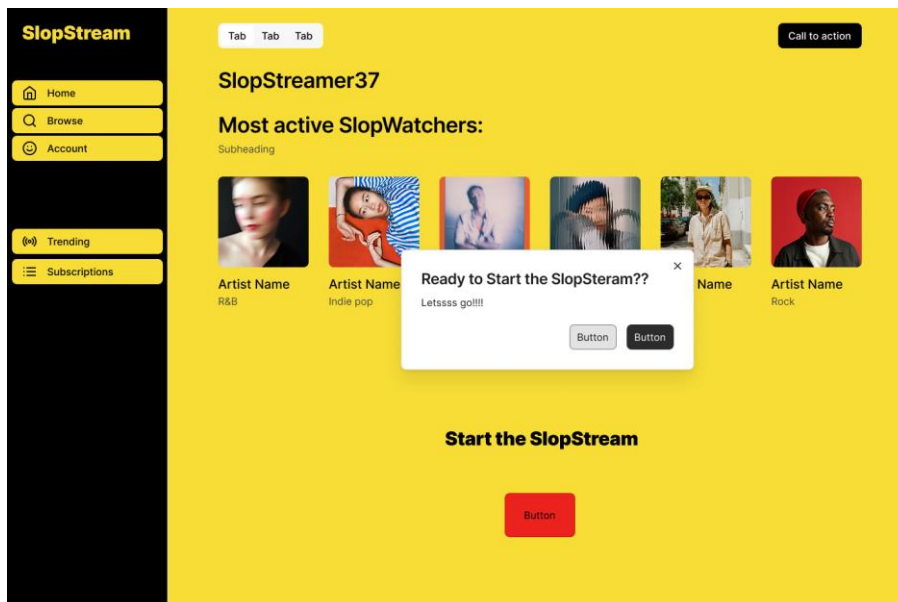
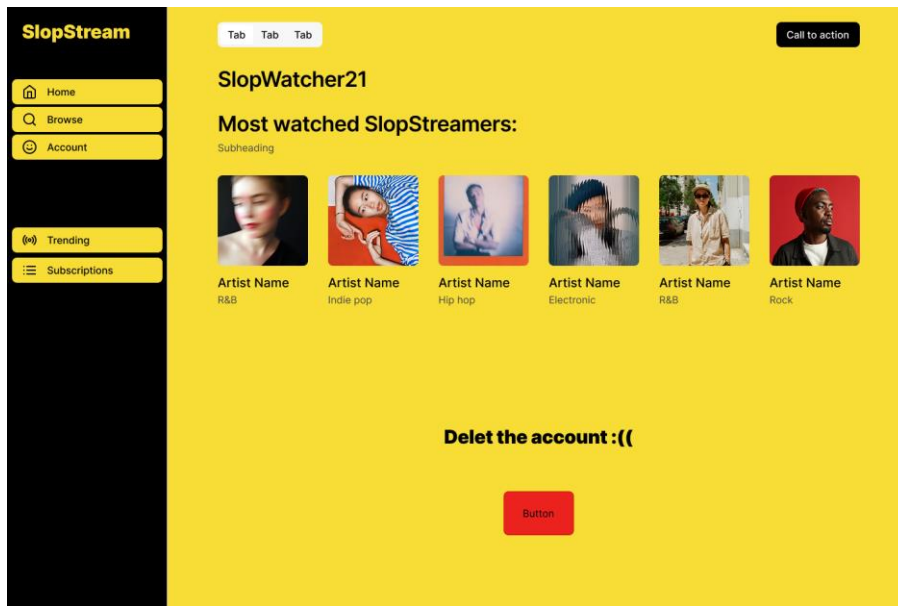
- Okno rejestracji/logowania:



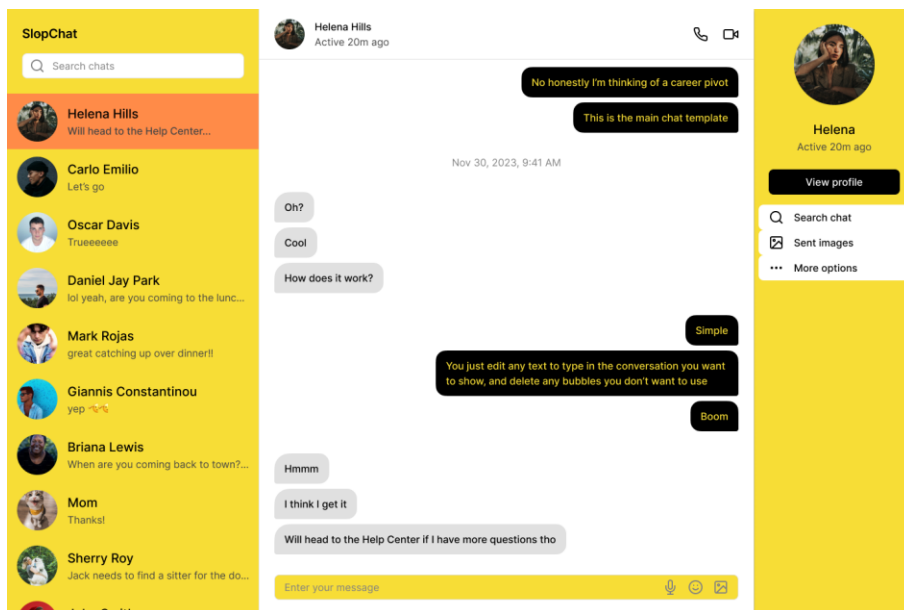
- **Okno strony użytkownika:**



- **Okno strony stramera:**



- Okno chatu:



6. Wymagania niefunkcjonalne dla systemu:

6.1. Oszacowanie wielkości bazy danych:

Przewidywana wielkość bazy danych, potrzebna dla funkcjonowania naszego systemu to ok. 23 TB przy założeniach:

Użytkownicy i profile: Zakładając 100 000 zarejestrowanych użytkowników (średnio 2 KB na profil wraz z ustawieniami, subskrypcjami i historią) - 200 MB.

Przechowywanie VOD: Archiwalne zapisy z transmisji (nie starsze niż 30 dni) (średnio 20 GB, przy 1 000 streamów miesięcznie) - 20 TB.

Transakcje i donacje: Logi subskrypcji, płatności, punktów kanału - 100 MB .

Wiadomości: historia wiadomości pomiędzy użytkownikami z ostatnich 30 dni(średnio 1 KB za wiadomość, przy 1 000 000 wiadomości) - 1 GB

6.2. Propozycja wymaganych czasów odpowiedzi:

Dla platformy streamingowej kluczowe są dwa aspekty: opóźnienie transmisji oraz czas odpowiedzi interfejsu API.

Opóźnienie strumienia wideo:

Maksymalne opóźnienie od momentu nadania sygnału przez streamera do wyświetlenia u widza: < 2-3 sekundy w celu zapewnienia płynnej interakcji na czacie oraz interakcji ze streamerem.

Czas odpowiedzi API systemu:

Operacje dynamiczne (wysłanie wiadomości na czacie, nadanie uprawnień moderatora): < 100 ms

Płatności i autoryzacja:

Obsługa zewnętrznych bramek płatniczych: < 5000 ms (zależne od zewnętrznych dostawców API).

6.3. Oszacowanie liczby i typów potrzebnych stanowisk pracy użytkowników systemu:

Streamerzy: od 100 do 200 streamerów, potrzebni aby dostarczać transmisję użytkownikom.

Typ: Komputer dostosowany pod transmitowanie/kamera (do transmisji spoza domu) – konieczne szybkie i stabilne połączenie aby przesyłać obraz w dobrej jakości.

Moderacja i Support: 30 pracowników po 5 na zmianę - konieczność funkcjonowania tej roli (24/7), aby zapewnić użytkownikowi pomoc, oraz dbać o etyczne treści na platformie.

Typ: Standardowe komputery biurowe z dwoma monitorami (jeden do podglądu streamu/czatu, drugi do panelu administracyjnego systemu i ticketów).

Zespół DevOps: od 2 do 5 pracowników, ich zadaniem jest monitorować na bieżąco stan systemu oraz poprawiać jego wydajność.

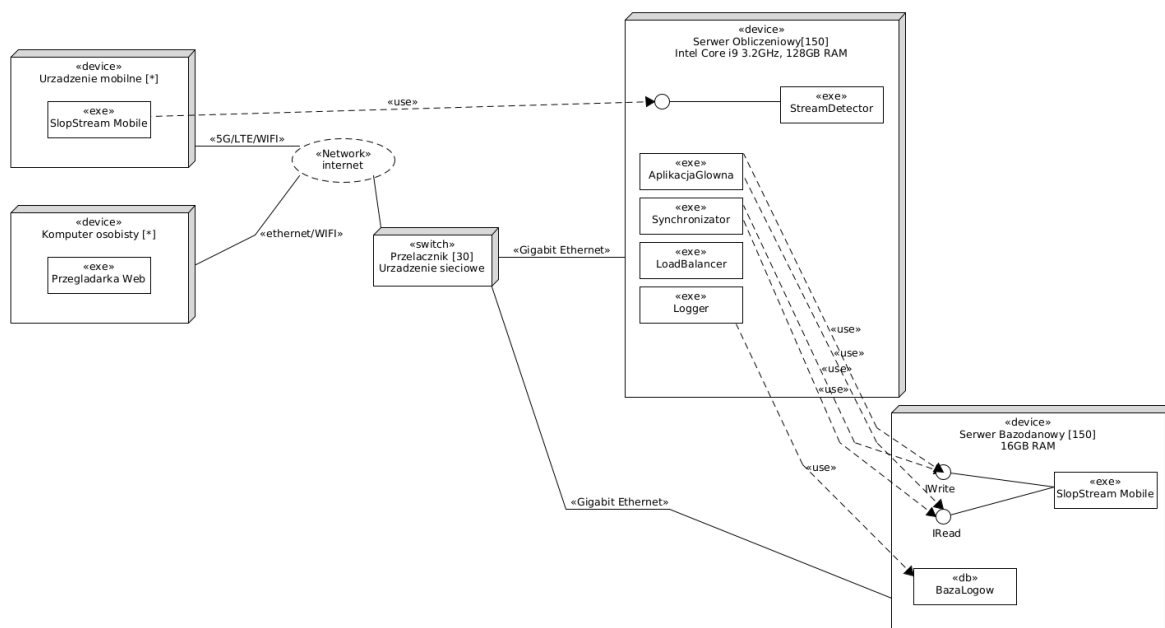
Typ: Stacje robocze z bezpiecznym, szyfrowanym dostępem (VPN/SSH) do infrastruktury chmurowej, systemów monitoringu

7. Propozycja technologii informatycznych, które mogą zostać wykorzystane do realizacji systemu:

Prezentowany diagram przedstawia architekturę fizyczną rozproszonego systemu SlopStream (30 serwerowni w 25 krajach). Infrastruktura została podzielona na cztery współpracujące warstwy:

- Klienta (aplikacje www i mobilne)
- Sieciowa (przełączniki i LoadBalancer)
- Obliczeniowa (150 serwerów aplikacji z modułami)
- Danych (150 serwerów bazodanowych)

Lokalna komunikacja opiera się na standardzie Gigabit Ethernet, co gwarantuje niskie opóźnienia (<100ms)



8. Propozycja planu pracy:

Plan pracy został podzielany na etapy, które reprezentują kluczowe składowe realizacji projektu.

Faza 1 – Inicjacja projektu:

Czas trwania: 2-4 tygodnie.

Na pierwszym etapie pracy nad projektem należy określić jego wizję, ustalić budżet oraz wstępny harmonogram.

Faza 2 – Analiza funkcjonalności biznesowych:

Czas trwania: 1-2 miesiące.

Podczas realizacji tego etapu, rozpatrywane są niezbędne funkcjonalności systemu jak i to w jaki sposób użytkownicy mają przeprowadzać z nim interakcję.

Faza 3 – Projekt Architektury:

Czas trwania: 1 miesiąc.

W trakcie trzeciej fazy podejmowane są decyzje dotyczące architektury systemu, zalicza się do tego narzędzia programistyczne, schemat budowy bazy danych, rodzaj komunikacja między komponentami czy dobór infrastruktury serwerowej.

Faza 4 – Budowa projektu:

Czas trwania: 2-4 miesiące.

Jest to najbardziej znaczący etap, ponieważ to w trakcie jego realizacji powstaje system. Przeprowadzane są sprinty, podczas, których wdrażane są kolejne elementy projektu. Ostatecznie powstaje pierwsza działająca wersja systemu.

Faza 5 – Testy:

Czas trwania: 4-6 tygodni.

Po stworzeniu działającej wersji projektu w poprzedni etapie, należy ją przetestować. Testy pokrywają takie aspekty jak bezpieczeństwo systemu, jego wydajność oraz jakość użytkowania. Testy są przeprowadzane na zmianę z wprowadzanie poprawek, aż do momentu uzyskania satysfakcjonującego efektu.

Faza 6 - Wdrożenie:

Czas trwania: 2 tygodnie.

Serwis zostaje po raz pierwszy udostępniony publicznie. Problemy, które nie zostały wychwycone podczas testów zostają naprawiane.

Faza 7 – Wsparcie:

Czas trwania: Do wyłączenia serwisu przez właścicieli.

Mimo tego, że produkt został wdrożony, a użytkownicy z niego korzystają, praca zespołu programistycznego trwa dalej. Razem ze wzrostem popularności system jest skalowany, naprawiane są błędy oraz dodawane kolejne funkcjonalności.

9. Analiza ryzyka projektu:

Techniczne

Przeciążenie infrastruktury – nagły napływ widzów podczas popularnego streamu powoduje awarię serwerów.

Wysokie opóźnienia streamu – opóźnienie między streamerem a czatem przekracza akceptowalny czas, niszcząc interakcję na żywo.

Ataki DDoS na platformę – celowe zablokowanie serwerów API lub czatu przez użytkowników z zewnątrz lub konkurencję.

Prawne

Prawa autorskie i DMCA – streamerzy puszczaają licencjonowaną muzykę lub filmy, co grozi platformie pozwami i problemami prawnymi.

Biznesowe

Brak rentowności – ogromne koszty transferu danych wideo przewyższają zyski z reklam, subskrypcji i donacji.

10. Kosztorys realizacji przedsięwzięcia (koszty projektu, oprogramowania, systemu, szkoleń, wdrożenia oraz konsultacji):

Poniższy kosztorys prezentuje szacunkowe nakłady finansowe potrzebne do realizacji, wdrożenia oraz przygotowania do startu rozproszonej platformy streamingowej SlopStream

Kategoria kosztów	Szczegółowy zakres i uzasadnienie	Szacunkowy koszt (PLN)
Koszty projektu i oprogramowania	Wynagrodzenie zespołu deweloperskiego (backend, frontend, mobile, QA, UI/UX)	1 500 000
Koszty systemu (infrastruktura sprzętowa)	Zakup sprzętu do wszystkich węzłów: 150 serwerów obliczeniowych, 150 serwerów bazodanowych oraz 30 przełączników sieciowych	3 650 000
Koszty sprzętu dla stanowisk pracy	Zakup stacji roboczych i monitorów dla personelu niezbędnego do startu systemu: 30 stanowisk dla zespołu moderacji i supportu oraz 5 stanowisk dla inżynierów DevOps	150 000
Koszty wdrożenia	Koszty instalacji i konfiguracji infrastruktury w 30 wynajętych przestrzeniach serwerowych. Konfiguracja komponentów LoadBalancer, Logger a także uruchomienie mechanizmów replikacji i synchronizacji baz danych	450 000
Koszty szkoleń	Przeprowadzenie specjalistycznych szkoleń dla inżynierów DevOps z zakresu zarządzania rozproszoną siecią w sytuacjach krytycznych oraz szkolenia stanowiskowe dla 30 moderatorów z obsługi paneli administracyjnych i modułu wsparcia	80 000

Koszty konsultacji	Audyty bezpieczeństwa kodu i infrastruktury, zewnętrzne testy penetracyjne, doradztwo prawne w zakresie licencji wideo, praw autorskich (DMCA) oraz globalnej ochrony danych osobowych.	120 000
PODSUMOWANIE	Całkowity koszt realizacji i uruchomienia przedsięwzięcia	5 950 000